

README

Primer borrador del README del proyecto. Basado en el análisis inicial (Issue 1), enfocado a SVR, y ampliado con la planificación de modelado, dashboard y producto.

(23/1/26 Rocio Perez Lopez)

Predicción de precios de coches usados (SVR) — EDA, optimización y dashboard

De mercado a producto: análisis reproducible + modelo explicable + demo interactiva.

1. Descripción general

Este repositorio recoge un proyecto de **regresión** para predecir el **precio** de coches usados a partir de datos reales (Cars.com). Entregamos:

- **Notebook(s)** en Google Colab: EDA, limpieza, feature engineering, entrenamiento y evaluación.
 - **Modelo SVR** con pipeline (preprocesado + estimador), validado con **K-Fold CV** y optimizado con **Optuna**.
 - **Aplicación en Streamlit** para predicción + recogida de feedback (predicción vs precio real).
 - **Dashboard analítico** (Power BI o Streamlit) con KPIs de mercado y panel de rendimiento del modelo.
-

2. Objetivos del proyecto

1. Entender el mercado de coches usados y los factores que más influyen en el precio.
2. Construir un modelo de regresión robusto con **SVR**, priorizando generalización y control de overfitting.
3. Validar con **cross-validation** y mantener un **test aislado** como auditoría final.
4. Optimizar hiperparámetros (C, epsilon, gamma, kernel) con **Optuna**, sin sobreajuste.
5. Productivizar el modelo con una **demo en Streamlit + dashboard** de análisis y evaluación.

3. Estructura del repositorio

- 📁 proyecto-7-grupo-4-prediccion-precios-coches-svr-dashboard/
 - └─ 📁 data/ # dataset original y dataset limpio
 - └─ 📁 notebooks/ # EDA + entrenamiento + evaluación
 - └─ 📁 src/ # utilidades (limpieza, features, training, etc.)
 - └─ 📁 app/ # Streamlit (predicción + feedback)
 - └─ 📁 dashboard/ # Power BI (.pbix) o assets (capturas)
 - └─ 📄 requirements.txt # dependencias
 - └─ 📄 README.md # este archivo
-

4. Tecnologías utilizadas

- **Python:** NumPy, pandas, scikit-learn
 - **Optimización:** Optuna
 - **Visualización:** matplotlib, seaborn
 - **Entorno:** Google Colab (recomendado) / Jupyter
 - **Producto:** Streamlit
 - **Dashboard:** Power BI (o versión equivalente en Streamlit)
 - **Control de versiones:** Git + GitHub
-

5. Cómo ejecutar el proyecto

Opción A — Google Colab (recomendado)

1. Clona el repositorio.
2. Abre el notebook principal en Colab (carpeta notebooks/).
3. Instala dependencias:
4. `pip install -r requirements.txt`
5. Ejecuta todas las celdas (EDA → limpieza → entrenamiento → evaluación).

Opción B — Local

1. Crea un entorno:

2. `python -m venv .venv`
3. `source .venv/bin/activate` # (Windows: `.venv\Scripts\activate`)
4. Instala dependencias y ejecuta `notebooks/app`.

Streamlit (demo)

`streamlit run app/Home.py`

6. Dataset y variables (EDA avanzado y “clasificación” de features)

Fuente: Kaggle — Used Car Price Prediction Dataset (Cars.com).

Tamaño: 4.009 registros.

Target: price (numérico, con alta dispersión y outliers típicos del mercado).

Tipología de variables (cómo las tratamos y por qué)

6.1 Variables numéricas (continuas / ordenadas)

- `model_year` → **ordinal** (edad del vehículo).
 - Rango: 1974–2024, con concentración fuerte en 2014–2024.
 - Riesgo: *efecto “corte” por cohortes* (mercado cambia por generaciones).
- `milage` → **numérica continua** (uso del vehículo).
 - Originalmente viene como texto ("110,000 mi."), requiere parsing.
 - Riesgo: outliers y distribución sesgada (cola larga).

6.2 Variables categóricas de baja / media cardinalidad (modelables con one-hot)

- `brand` (57 categorías) → **categórica nominal**.
 - Aporta señal de posicionamiento (premium vs generalista).
 - One-hot viable (no explota dimensiones).
- `fuel_type` (7 categorías, 4% nulos) → **categórica nominal**.
 - Suele capturar “economía vs tecnología vs normativa” (gasolina/híbrido/eléctrico).
- `transmission` (62 categorías) → **categórica nominal**.
 - Aquí hay granularidad comercial (“8-Speed A/T”, etc.).
 - Se puede mantener o agrupar (según impacto).
- `accident` (2 categorías, ~3% nulos) → **binaria**.

- Señal fuerte: historial de accidente suele penalizar precio.

6.3 Variables categóricas de alta cardinalidad (peligro de one-hot masivo)

- model (1898 valores únicos) → **alta cardinalidad**.
 - One-hot completo = riesgo de dimensionalidad alta + sparse extremo.
 - Decisión de diseño (clave en SVR): preferimos **frequency encoding** o agrupar raros a Other por umbral de frecuencia.
 - Razón: SVR (sobre todo con RBF) es sensible a la geometría del espacio y al coste computacional.
- engine (1146 valores únicos) → texto semiestructurado.
 - En vez de one-hot, extraemos señal numérica: litros, cilindros, horsepower, turbo.
 - Esto suele dar más retorno que “pelearse” con el kernel sin mejorar features.

6.4 Variables estéticas (alto ruido, señal débil, alta cardinalidad)

- ext_col (319 valores) y int_col (156 valores)
 - Pueden influir en casos concretos, pero suelen introducir ruido, rarezas de naming y colisiones (“Black”, “Jet Black”, etc.).
 - En SVR pueden perjudicar por dimensionalidad. Las tratamos con criterio:
 - normalización de texto + agrupación a colores base, o
 - eliminación si no mejoran métricas / generalización.

6.5 Variable problemática (calidad de dato)

- clean_title
 - En el dataset aparece como true o nulo, sin falsos.
 - Esto limita su valor predictivo tal y como está: puede acabar siendo casi un “indicador de missingness”.
 - Decisión: imputación explícita (Unknown) o binarización (is_clean_title_known).

7. Proceso de limpieza y transformación

- **Normalización de tipos**
 - price: de "\$15,000" a entero/float.
 - milage: de "110,000 mi." a entero.
 - model_year: a entero.
- **Tratamiento de nulos**
 - fuel_type (≈4%): imputación a Unknown o moda (según impacto).
 - accident (≈3%): imputación a Unknown (evita inventar "None reported").
 - clean_title (≈15% nulos): tratamiento explícito para no esconder el patrón.
- **Ingeniería de variables (barata + intención)**
 - Desde engine: extracción de litros, cilindros, HP, turbo.
 - Transformación de target: entrenamiento con $\log_{10}(\text{price})$ si estabiliza outliers (y posterior inversión para mostrar precio).
- **Control de calidad / filtrado**
 - Reglas de plausibilidad: model_year razonable, milage ≥ 0 , precios extremos revisados.
 - Duplicados: revisión y eliminación si aplica (misma ficha).

8. Modelado (SVR + defendible)

8.1 Por qué SVR

En vez de "una recta", en múltiples dimensiones buscamos una **superficie** que capture la tendencia con un **margen de tolerancia**. SVR trabaja exactamente en esa lógica: minimiza error manteniendo un **ϵ -tubo** (tolerancia) y controlando complejidad con **C**.

8.2 Pipeline recomendado

- Preprocesado:
 - Escalado numérico obligatorio: StandardScaler o RobustScaler.
 - Categóricas: one-hot solo donde aporta valor sin explotar dimensiones.
 - Alta cardinalidad (model, engine): encoding/feature extraction.
- Entrenamiento:

- Baseline: Linear/Ridge (para comparar, no para “decorar”).
- SVR: kernel (rbf/linear), C, epsilon, gamma (si RBF).

8.3 Validación y control de overfitting

- **K-Fold CV** para estimar rendimiento con menos sesgo.
- **Test aislado** como auditoría final: no se usa para decisiones.
- Overfitting < 5% (criterio del proyecto), medido como:
 - $\text{gap} = (\text{RMSE}_{\text{val}} - \text{RMSE}_{\text{train}}) / \text{RMSE}_{\text{train}}$

8.4 Explicabilidad en SVR

- SVR no da “feature importance” nativa.
 - Solución:
 - **Permutation importance** sobre el pipeline final.
 - Gráficos de **predicción vs real** y **residuos** para detectar sesgos.
-

9. Análisis y visualizaciones

Preguntas que resolvemos (mercado + modelo):

- ¿Cómo cae el precio cuando sube el kilometraje?
- ¿Cuánto pesa el año del coche frente a otros factores?
- ¿Qué marcas sostienen mejor el valor (mediana, no media)?
- ¿Dónde se equivoca más el modelo: precios altos, coches antiguos, outliers?

Gráficos clave (por qué están y no otros):

- Distribución de price (y log(price)) → para entender sesgo y outliers.
 - Scatter price vs milage + tendencia → relación directa y no lineal.
 - Scatter price vs model_year + tendencia → depreciación/cohortes.
 - Correlaciones (numéricas) + análisis por segmentos (brand/fuel) → contexto de negocio.
-

10. Dashboard final (Power BI o Streamlit)

El dashboard responde a dos cosas:

1. **Mercado:** qué factores mueven el precio y cómo se segmenta.
2. **Modelo:** qué tan bien predice SVR y dónde falla.

Página 1 — Visión general del mercado

KPIs (arriba): precio medio, precio mediano, nº vehículos, año medio, km medio.

Gráficos:

- Precio vs kilometraje (scatter + tendencia)
- Precio vs año (scatter + tendencia)
- Top 10 marcas por precio mediano (barras)
- Distribución por fuel type (barras o donut, solo uno)

Filtros (panel lateral fijo): marca, año (slider), fuel, transmisión, accident.

Página 2 — Modelo y predicción (SVR)

KPIs del modelo: RMSE (test), MAE (test), R^2 (test), gap overfitting (%).

Gráficos:

- Predicción vs real (con diagonal $y=x$)
- Residuos (error vs precio real)
- Permutation importance (Top 10)

Captura final:

11. Metodología de trabajo (Agile orientado a entrega)

Este proyecto se ha trabajado con un enfoque **Agile** orientado a entregar valor en una semana: primero aseguramos una base sólida (datos + baseline), después afinamos el modelo (SVR + validación), y cerramos con producto (app + dashboard + documentación). El objetivo no ha sido “hacerlo todo”, sino **tomar decisiones defendibles** y dejar un entregable reproducible.

Equipo y roles

- **Scrum Master:** Rocío Pérez López
- **Product Owner:** Yeray Mata Rodríguez
- **Data Analyst:** Jaime Amuedo Hidalgo

Aunque los roles están definidos, el criterio fue simple: **cualquier decisión que afecte a métrica, interpretabilidad o producto se discute y se registra.**

Flujo de trabajo

Organizamos el trabajo con un backlog priorizado y un tablero tipo Kanban, con un flujo pensado para revisión real (no solo “hecho/no hecho”):

Pendiente → Listo para empezar → En progreso → En revisión → Integrado

- **En revisión** implica revisión técnica y de consistencia (métricas, gráficos, supuestos, naming, reproducibilidad).
- **Integrado** significa que está en rama principal y documentado (README + notebook).

Además, hicimos checkpoints cortos para alinear:

- criterios de limpieza,
- definición de overfitting (<5%),
- qué se considera “explicabilidad” en SVR (residuos + permutación),
- y qué debe contestar el dashboard (mercado + modelo).

Plan de sprints (1 semana, 3 sprints)

Para encajar el plazo, dividimos la semana en tres tramos con entregables cerrados. Cada sprint termina con algo “mostrable”.

Sprint 1 — Base técnica + decisiones de datos (Días 1–2)

Entrega: dataset listo + EDA con hipótesis + baseline comparativo.

- Validación del dataset, revisión de cardinalidades y calidad (nulos, categorías raras, campos semiestructurados).
- Limpieza mínima pero consistente: parsing de price y milage, tipos correctos y reglas de plausibilidad.
- EDA orientado a decisiones (no solo gráficos):
 - por qué usamos **mediana** además de media (outliers),
 - qué hacemos con alta cardinalidad (model, engine),
 - riesgos de variables estéticas (ext_col, int_col).
- Baseline (Linear/Ridge) como referencia de rendimiento y para justificar SVR con métricas.

- Primer boceto de dashboard: preguntas, KPIs y filtros (en formato “wireframe”).

Sprint 2 — SVR + validación + explicabilidad (Días 3–5)

Entrega: pipeline completo con SVR + Optuna + informe de rendimiento.

- Pipeline de preprocesado + entrenamiento con SVR (escalado obligatorio y encoding controlado).
- Validación **K-Fold CV** para seleccionar hiperparámetros sin sesgo por un único split.
- Optimización con Optuna con criterio de negocio/técnico:
 - objetivo por RMSE/MAE,
 - penalización si el gap de overfitting supera el umbral.
- Informe de explicabilidad “honesto” para SVR:
 - predicción vs real,
 - residuos,
 - **permutation importance** (sustituto razonable de feature importance).
- Test aislado como auditoría final (sin usarlo para tuning).

Sprint 3 — Producto + dashboard + cierre (Días 6–7)

Entrega: demo en Streamlit + dashboard final + README + presentación.

- Streamlit como producto:
 - inputs con validaciones (año razonable, km no negativos, etc.),
 - predicción en tiempo real,
 - recogida opcional de feedback (predicho vs real) a CSV/SQLite.
 - Dashboard final (Power BI o Streamlit):
 - Página Mercado (KPIs + segmentación),
 - Página Modelo (métricas + sesgos + permutación).
 - Cierre de documentación:
 - README (reproducibilidad, decisiones, métricas),
 - capturas del dashboard y resultados,
 - estructura final del repo y requisitos.
-

Cómo encaja con los requisitos del proyecto

- **Regresión funcional:** target price y modelo SVR entrenado.
- **EDA + visualizaciones:** mercado + relaciones clave.
- **Overfitting <5%:** criterio explícito y medido.
- **Productivización:** Streamlit + dashboard.
- **Explicación del rendimiento:** residuos + predicción vs real + permutation importance.
- **Nivel medio:** K-Fold + Optuna + recogida de feedback/datos nuevos.

Nuestra meta fue que cualquiera pueda clonar el repositorio, ejecutar el notebook y entender por qué el modelo es el que es. En vez de buscar el ‘mejor algoritmo’, priorizamos decisiones trazables: datos bien tratados, validación limpia y un dashboard que cuente la historia completa (mercado + modelo).

12. Conclusiones (lo que esperamos demostrar)

- El precio no se explica por una sola variable: hay interacción clara entre **año, km, marca y motor**.
 - La **mediana** es más fiable que la media para comparar segmentos por presencia de outliers.
 - SVR puede capturar no linealidad, pero su rendimiento depende más del **preprocesado y features** que de “probar kernels al azar”.
 - El análisis de **residuos** es lo que nos dice si el modelo es útil en producción o solo “pasa métricas”.
 - La app + feedback convierte el proyecto en algo vivo: no solo entrenamos, también medimos cómo se comporta con uso real.
-

13. Aprendizajes

- Diseñar un pipeline pensando en el modelo (SVR necesita escalado y control de dimensionalidad).
- EDA orientado a decisiones: qué variables merecen quedarse, cuáles añaden ruido.
- Validación cruzada como estándar cuando el dataset no es enorme.

- Explicar modelos sin “feature importance fácil”: permutación + residuos como alternativa sólida.
 - Convertir un notebook en producto: validaciones, feedback y métricas observables.
 - **La limpieza no es un paso previo, es un proceso iterativo**
El trabajo en Power Query y en Python nos confirmó que la limpieza no se “cierra” al principio. Volvimos a transformar datos después de visualizar, modelar o validar hipótesis. La capacidad de rehacer pasos sin romper el pipeline es una habilidad clave en proyectos reales.
 - **Modelar bien importa más que el gráfico final**
Un dashboard atractivo no compensa un modelo de datos incorrecto. Dedicamos tiempo a pensar relaciones, cardinalidades y estructura (modelo estrella) antes de visualizar. Esto reduce ambigüedades, evita errores silenciosos y hace que el análisis escale cuando crecen los datos o las preguntas.
 - **Normalización y desnormalización son decisiones, no dogmas**
Entendimos las formas normales como guías, no como reglas rígidas. Normalizamos para mantener integridad y coherencia, y desnormalizamos cuando el objetivo analítico o el modelo de ML lo requería. Documentar el “por qué” de cada decisión es tan importante como la decisión en sí.
 - **Un modelo puede pasar métricas y aun así no ser usable**
El análisis de residuos nos enseñó que un buen RMSE medio no garantiza buen comportamiento en todos los segmentos. Detectamos rangos donde el modelo falla sistemáticamente y asumimos que evaluar sesgos es parte del trabajo, no un extra opcional.
 - **La validación es una defensa contra el autoengaño**
Separar correctamente train, validación y test, y no tocar el test durante el ajuste, no es una formalidad académica: es una práctica profesional para evitar optimismo artificial. La validación cruzada nos permitió estimar rendimiento con menos varianza en datasets no masivos.
 - **Las herramientas no sustituyen el criterio**
Power BI, scikit-learn u Optuna automatizan mucho, pero no toman decisiones por nosotros. Elegir features, definir métricas, decidir cuándo parar el tuning o qué variable eliminar sigue siendo trabajo humano basado en criterio analítico.
 - **Pensar el proyecto como producto cambia las decisiones técnicas**
Diseñar Streamlit con validaciones, feedback y recogida de datos reales nos obligó a pensar más allá del notebook. El modelo deja de ser un ejercicio puntual y pasa a ser un sistema que puede fallar, aprender y mejorar con el uso.
 - **Documentar decisiones es parte del deliverable**
No solo entregamos resultados, sino también el razonamiento detrás de cada elección: encoding, métricas, filtros del dashboard, estructura Agile. Esto permite que otra persona (técnica o no) entienda, mantenga y evolucione el proyecto.
-

14. Referencias

- **Dataset**
- Kaggle: *Used Car Price Prediction Dataset (Cars.com)* [*Used Car Price Prediction Dataset*](#)
- **Brief / requisitos del proyecto**
- Factoria F5: *DA – Project Regression (entregables, métricas y niveles)* [*Factoria-F5-madrid/DA-Project-Regression*](#)
- **Documentación técnica**
- scikit-learn: SVR, pipelines, preprocessing
- Optuna: hyperparameter optimization